

Cartridge & Cloud - Technical Design Document

Arquitectura objetivo Unity/C# desde proyecto vacío

VRM Games / Blas Luis Rocha González

21/06/2026

1. Propósito y estado
2. Stack recomendado
3. Principios de arquitectura
4. Capas
 - 4.1 Domain
 - 4.2 Application
 - 4.3 Infrastructure
 - 4.4 Presentation
 - 4.5 Content
5. Contextos funcionales
6. Servicios e interfaces iniciales
7. Flujo de arranque
8. Escenas
9. Comunicación
10. Simulación temporal
11. Grid, construcción y navegación
12. Inventario y transacciones
13. Clientes y tareas
14. Persistencia
15. Rendimiento
16. Seguridad técnica y fallos
17. Estructura de carpetas propuesta
18. Assembly Definitions
19. Riesgos técnicos principales
20. Decisiones conceptuales relevantes
 27. Sistemas actuales reutilizables
 28. Sistemas nuevos y priorización técnica
 - Sistemas identificados
 - Fases recomendadas
 34. Próximo objetivo conceptual y de preproducción
 - Objetivo del vertical slice
 - Alcance aprobado
 - Criterios de aceptación
 - Definition of Done
 - Secuencia posterior aprobada
21. Criterio de aceptación del TDD

Proyecto: Cartridge & Cloud
Título técnico provisional: Cartridge & Cloud
Plataforma inicial: PC / Steam
Motor previsto: Unity 6 LTS / C#
Versión del documento: v0.3
Estado: Preproducción reiniciada desde cero
Fuente conceptual: Enfoque_v0.6.md

Regla de interpretación: el proyecto se documenta como una preproducción nueva. No se presupone código, escenas, managers, datos, builds ni sprints implementados. Todo elemento técnico descrito es una especificación o recomendación hasta que exista evidencia de implementación y QA.

1. Propósito y estado

Este TDD define una arquitectura objetivo inicial. No documenta código existente. Las decisiones deben validarse mediante spikes técnicos y ADR antes de considerarse definitivas.

2. Stack recomendado

Área	Decisión inicial
Motor	Unity 6 LTS; versión exacta congelada en Sprint 0
Render	URP 3D, iluminación contenida y assets estilizados
Lenguaje	C# con nullable warnings donde sea viable
UI	Canvas/uGUI + TextMeshPro
Input	Unity Input System
Navegación	AI Navigation / NavMesh
Cámara	Cámara propia o Cinemachine tras spike
Localización	Unity Localization desde que exista UI estable
Testing	Unity Test Framework + pruebas manuales
Persistencia	JSON versionado inicialmente; abstracción de repositorio
Plataforma	Windows x64; Steam más adelante

3. Principios de arquitectura

- Dominio independiente de MonoBehaviour cuando sea razonable.
- Composición y servicios pequeños antes que un GameManager monolítico.
- Datos estáticos editables; estado runtime serializable.
- UI basada en comandos/resultados y eventos de solo lectura.
- Persistencia por snapshots coherentes en puntos seguros.
- Simulación determinista donde ayude a QA y guardado.
- No usar singletons como dependencia implícita general.
- No guardar referencias a escenas o ScriptableObjects dentro del save.

4. Capas

4.1 Domain

Entidades, value objects, estados y reglas puras: dinero, inventario, pedidos, clientes, empleados, transacciones, jornadas, proyectos editoriales y plataforma.

4.2 Application

Casos de uso y orquestación: colocar mueble, confirmar pedido, abrir tienda, procesar venta, cerrar día, guardar, contratar, publicar, lanzar plataforma.

4.3 Infrastructure

Unity adapters, persistencia, reloj, aleatoriedad, navegación, archivos, localización, audio, telemetría y futuras APIs.

4.4 Presentation

Controladores de UI, presenters/view-models, feedback en mundo, cámara y tutorial.

4.5 Content

ScriptableObjects o assets de configuración para productos, muebles, clientes, proveedores, eventos y balance.

5. Contextos funcionales

Contexto	Responsabilidad
Session	Nueva partida, carga, pausa y flujo de escenas
World	Grid, parcela, zonas, puertas, interacción
Construction	Colocación, validación, coste y refund
Inventory	Unidades, contenedores, reservas y transferencias
Supply	Proveedores, pedidos y entregas
Customers	Perfiles, necesidades, rutas, satisfacción y compra
Checkout	Cola, transacciones y caja
Time	Reloj, velocidades, apertura, cierre y calendario
Economy	Libro mayor, impuestos, costes y reportes
Employees	Candidatos, tareas, fatiga, salarios y salida
Research	Nodos, requisitos, tiempo y desbloques
Online	Catálogo, pedidos, packing, envío y devoluciones
Publishing	Propuestas, contratos, hitos y liquidaciones
Development	Brief, prototipos, backlog, builds y lanzamiento
Platform	Usuarios, licencias, catálogo y pagos
Infrastructure	Capacidad, servicios, incidencias y costes
Market	Competidores, segmentos, tendencias y acciones

6. Servicios e interfaces iniciales

```
IGameClock
IRandomService
ISaveRepository
IContentCatalog
IPathingService
IGridService
ITransactionLedger
INotificationService
ILocalizationService
IAudioService
```

Las implementaciones Unity se inyectan o registran durante Bootstrap. Los tests pueden utilizar dobles deterministas.

7. Flujo de arranque

```
Bootstrap scene
-> carga configuración global
-> registra servicios
-> inicializa localización y opciones
-> abre Main Menu
-> New/Load
-> carga Store scene
-> crea GameSession
-> restaura o genera estado
-> habilita simulación
```

8. Escenas

- `Bootstrap` : servicios persistentes mínimos.
- `MainMenu` : navegación y configuración.
- `Store` : escena principal de juego.
- `TestLab` : pruebas aisladas de grid, clientes, UI e inventario.
- `Loading` : opcional si la transición lo requiere.

9. Comunicación

Los sistemas publican eventos de dominio inmutables: `ProductReserved` , `SaleCompleted` , `StoreClosed` , `OrderDelivered` , `EmployeeHired` . La UI se actualiza mediante modelos de lectura. Las operaciones que pueden fallar devuelven resultados tipados con código, mensaje localizable y datos.

10. Simulación temporal

El reloj produce ticks de simulación a paso fijo independiente de FPS. Las velocidades cambian frecuencia/escala de ticks, no la velocidad de animaciones. Sistemas costosos se distribuyen por lotes y eventos, evitando recorrer todas las entidades cada frame.

11. Grid, construcción y navegación

- Grid lógico de 0,5 m.
- Huellas enteras por orientación.
- Validación de colisión, límites, acceso e interacción.
- Separación entre representación visual y ocupación lógica.
- Rebake o actualización controlada de navegación tras cambios.
- Los agentes deben recuperar rutas fallidas y liberar reservas.

12. Inventario y transacciones

Cada unidad tiene un único propietario lógico y estado. Las transferencias son operaciones atómicas. El libro mayor registra toda variación monetaria. Las ventas y devoluciones usan IDs idempotentes para evitar duplicados al guardar/cargar.

13. Clientes y tareas

La IA se modela como máquina de estados o behaviour graph pequeño. Las decisiones comerciales se calculan al evaluar producto; el movimiento se delega a NavMesh. Las tareas de empleados usan cola priorizada, reservas de recursos y políticas de interrupción seguras.

14. Persistencia

- `saveVersion` obligatorio.
- Snapshot raíz con subestados por contexto.
- IDs estables y valores primitivos.
- Normalización y migración al cargar.
- Guardado manual solo en estados seguros.
- Autosave al cerrar jornada.
- Escritura temporal + reemplazo atómico.
- Copia de seguridad del slot anterior.

15. Rendimiento

Presupuestos iniciales del vertical slice:

Métrica	Objetivo
FPS a 1080p	60
Clientes simultáneos	8, escalable
Allocations por frame	Próximas a cero durante simulación estable
Tiempo de cierre/guardado	< 2 s en equipo de desarrollo
Tiempo de carga	< 5 s en vertical slice

16. Seguridad técnica y fallos

Las excepciones recuperables generan logs y feedback. Nunca se ignoran fallos de guardado, reservas huérfanas, transacciones duplicadas ni IDs faltantes. Los datos inválidos deben fallar en herramientas de editor o validación de build.

17. Estructura de carpetas propuesta

```
Assets/_Project/
  Art/ Audio/ Content/ Prefabs/ Scenes/ UI/
  Code/
    Domain/ Application/ Infrastructure/ Presentation/
    Editor/ Tests/
  Settings/
  Documentation/
  Builds/
```

18. Assembly Definitions

Separar al menos: Domain, Application, Infrastructure.Unity, Presentation, Editor y Tests. Domain no referencia UnityEngine salvo tipos estrictamente necesarios y justificados.

19. Riesgos técnicos principales

Riesgo	Mitigación
NavMesh tras construcción	Spike temprano y límites de rebake
Inventario duplicado	Operaciones atómicas e invariantes automáticas

Riesgo	Mitigación
IA atascada	Timeout, replanificación y estados de recuperación
Save demasiado grande	IDs, datos compactos y snapshots por contexto
UI acoplada	Casos de uso y modelos de lectura
Scope de sistemas avanzados	Implementación por capas y MVP por sistema

20. Decisiones conceptuales relevantes

27. Sistemas actuales reutilizables

Los sistemas candidatos a conservar o adaptar son:

- TimeManager .
- EconomyManager .
- SaveManager .
- DataManager .
- GameManager .
- Arquitectura de managers.
- IDs estables.
- ScriptableObjects.
- Guardado versionado.
- Datos de proyectos.
- Reviews.
- Ventas.
- Publicación.
- Investigación como concepto.
- Herramientas de debug.
- QA.
- Trazabilidad.
- Identidad de interfaz.

La auditoría se realizará después de:

1. Cerrar la nueva dirección.
2. Actualizar la documentación principal.
3. Definir el primer prototipo.
4. Establecer la arquitectura objetivo.

Estado del apartado: APLAZADO.

La auditoría no debe comenzar todavía. Más adelante cada sistema se clasificará como reutilizable, adaptable, reemplazable o descartable.

28. Sistemas nuevos y priorización técnica

Sistemas identificados

- Movimiento libre.
- Cámara orbital.
- Cuadrícula lógica.
- Pathfinding.
- Colocación.
- Construcción modular.
- Interacción.
- Estanterías y vitrinas.
- Productos.
- Inventarios.
- Pedidos.
- Proveedores.
- Clientes.
- Mostrador.
- Colas.
- Pago.
- Día y economía.
- Empleados.
- Investigación.
- Ampliaciones.
- Puestos informáticos.
- Tiers.
- Electricidad.
- Productos usados.
- Resumen diario.

Fases recomendadas

Fase R0 — Cierre conceptual

- Resolver decisiones del primer prototipo.
- Actualizar GDD, TDD y modelo de datos.
- Definir alcance y criterios de aceptación.

Fase R1 — Auditoría y rebase técnico

- Auditar código existente.
- Diseñar arquitectura objetivo.
- Crear rama o proyecto de rebase.
- Conservar solo sistemas compatibles.

Fase P1 — Mundo y control

Dependencias iniciales:

- Cuadrícula lógica.
- Suelo y zonas.
- Movimiento libre.
- Cámara.
- Pathfinding.
- Interacción básica.

Fase P2 — Construcción de tienda

Depende de P1:

- Colocación.
- Rotación.
- Huellas.
- Validación.
- Estanterías.
- Vitrinas.
- Mostrador.
- Almacén.

Fase P3 — Productos y logística

Depende de P2:

- Datos de producto.
- Stock.
- Proveedores básicos.
- Pedidos.
- Recepción.
- Reposición.
- Visuales de llenado.

Fase P4 — Cliente y venta

Depende de P2 y P3:

- Spawn.
- Perfiles básicos.

- Navegación.
- Selección.
- Reserva.
- Cola.
- Pago.
- Salida.
- Satisfacción inicial.

Fase P5 — Día, economía y persistencia

Depende de P3 y P4:

- Apertura y cierre.
- Escala temporal.
- Resumen diario.
- Gastos.
- Beneficios.
- Guardado y carga.

Fase P6 — Empleados

Depende de P2–P5:

- Contratación básica.
- Tareas.
- Prioridades.
- Salario.
- Experiencia.
- Cansancio.

Fase P7 — Investigación y ampliaciones

Depende de P5:

- Puntos.
- Árbol.
- Tiempos.
- Costes.
- Nuevas zonas.
- Requisitos.

Fase P8 — Puestos informáticos

Depende de P2, P4, P5 y P7:

- Setup por slots.
- Tiers.
- Uso.

- Cobro.
- Electricidad.
- Productos usados.

Fases posteriores

- Reservas.
- Comercio online.
- Distribución.
- Publishing.
- Desarrollo.
- Plataforma.
- Servidores.

Estado del apartado: PARCIALMENTE DEFINIDO.

La prioridad y las dependencias están establecidas. Los sprints concretos se numerarán después de la auditoría técnica y la estimación del nuevo prototipo.

34. Próximo objetivo conceptual y de preproducción

Las decisiones conceptuales 1-26 están cerradas. El primer vertical slice queda aprobado formalmente como objetivo de validación de la nueva dirección.

Objetivo del vertical slice

Demostrar que la fantasía principal de la tienda física funciona antes de desarrollar empleados, comercio online, publishing, desarrollo interno o plataforma.

Debe validar:

1. Control y cámara.
2. Construcción física del local.
3. Pedido, recepción, almacén y reposición.
4. Clientes, selección de productos, colas y cobro.
5. Precios, stock, gastos y ventas.
6. Apertura, cierre y progresión diaria.
7. Guardado y carga íntegros.

Alcance aprobado

Contexto inicial

- Tienda 10×10 dentro del mapa lógico de 100×100 .
- Capital: 20.000 €.
- Local alquilado y vacío.

- Mobiliario Tier E.
- Un proveedor general.
- Sin empleados.
- El jugador realiza manualmente todas las tareas.

Sistemas incluidos

- Movimiento por clic y NavMesh.
- Cámara orbital de 360° y zoom 5–18 m .
- Construcción sobre cuadrícula y rotación de 90°.
- Mostrador, estantería, vitrina, almacén y recepción.
- Catálogo mínimo de 12 productos y 6 familias.
- Proveedor, pedidos, transporte y entrega.
- Recepción, cajas, almacén, asignación y reposición.
- Precios entre 50-150 % del recomendado.
- Cuatro arquetipos de cliente.
- Máximo inicial de 8 clientes simultáneos.
- Cola y cobro manual por el jugador.
- Apertura desde las 08:00, cierre manual desde las 20:00 y obligatorio a las 22:00.
- Economía diaria y semanal.
- Resumen diario y semanal.
- Impuesto al séptimo día.
- Guardado manual con tienda cerrada.
- Autoguardado tras el resumen.
- Tres slots.
- Localización ES/EN.

Contenido mínimo

Elemento	Mínimo
Escenario	1
Tienda	1
Proveedor	1
Productos	12
Familias	6
Arquetipos de cliente	4
Muebles funcionales	5
Días simulables	7 o más
Velocidades temporales	5
Clientes simultáneos	8

Elemento	Mínimo
Idiomas	ES/EN
Slots	3

Fuera del alcance

- Empleados.
- Investigación funcional.
- Ampliaciones del complejo.
- Puestos informáticos.
- Reservas.
- Eventos de tienda.
- Comercio online.
- Publishing.
- Desarrollo interno.
- Plataforma y servidores.
- Usados y retro.
- Devoluciones y robos.
- Competidores.
- Marketing avanzado.
- Steam, logros, Steam Cloud o demo pública.
- Arte y audio finales.

Criterios de aceptación

El slice debe permitir completar al menos **7 días** y cumplir:

- El jugador llega a cualquier punto accesible sin atravesar obstáculos.
- La cámara no interfiere con UI ni colocación.
- No se puede solapar mobiliario ni bloquear accesos obligatorios.
- Los pedidos llegan una sola vez y con las cantidades correctas.
- El stock cumple siempre:

`Almacén + exposición + reservas temporales = stock total`

- El precio afecta al interés de compra.
- Los clientes no duplican reservas ni quedan bloqueados indefinidamente.
- Cada venta se registra una sola vez.
- El cierre elimina clientes y reservas temporales de forma segura.
- El saldo solo cambia mediante movimientos registrados.
- El impuesto se aplica únicamente al beneficio semanal positivo.
- Guardar, cerrar y cargar conserva dinero, inventario, muebles, pedidos, precios y semana fiscal.

- No existen bugs bloqueantes ni excepciones recurrentes.
- Rendimiento objetivo: 60 FPS a 1920 × 1080 con 8 clientes y tienda equipada.

Definition of Done

1. Sistemas incluidos implementados.
2. Criterios obligatorios validados.
3. Partida de siete días completada.
4. Guardado y carga sin diferencias de estado.
5. Sin bugs bloqueantes o críticos abiertos.
6. Bugs importantes restantes documentados.
7. Documentación técnica actualizada.
8. Trazabilidad registrada.
9. Build interna de Windows generada.
10. Informe de validación producido.

Secuencia posterior aprobada

Consolidar decisiones 1–26

- actualizar GDD, TDD, modelo de datos, UX, arte, audio, QA, roadmap y planes afectados
- auditar el código actual
- decidir qué sistemas se reutilizan, refactorizan o sustituyen
- diseñar la nueva arquitectura
- crear el roadmap por sprints del vertical slice
- implementar y validar el slice
- desarrollar las etapas avanzadas según el orden de progresión aprobado

Estado del apartado: COMPLETO A NIVEL CONCEPTUAL.

El alcance, las exclusiones, los criterios de aceptación y la Definition of Done del vertical slice están aprobados.

21. Criterio de aceptación del TDD

El TDD se considera listo para implementación cuando los spikes de cámara, grid/NavMesh, inventario y guardado confirman viabilidad, y cuando el Modelo de Datos y QA Matrix referencian las mismas entidades y estados.